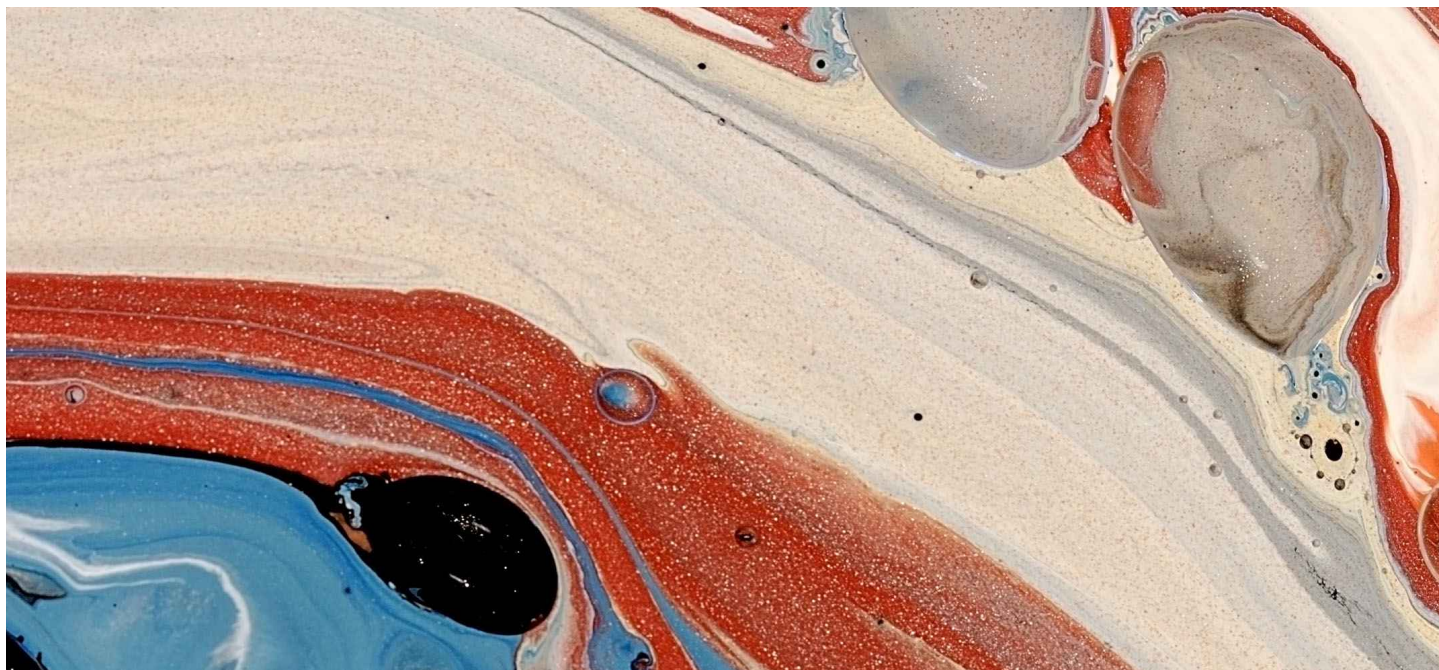




测试业务1-Java控制台实现教务管理系统

1. 项目背景以及介绍

在高校教育管理中，教务管理系统扮演着关键角色。为了更好地管理学生、课程、成绩等信息，我们决定开发一个简单而实用的教务管理系统。该系统可以由管理员管理多个班级和学生，实现学生的增删改查等基本操作。

2. 项目整体描述

这个Java控制台教务管理系统是专为管理员设计的，旨在简化学生信息和班级管理。其主要特点包括简洁易用的控制台界面、本地存储数据、以及基于文本文件的数据存储方式。

管理员通过控制台与系统进行交互，可以进行以下功能操作：

1. 学生信息管理：管理员可以添加、删除、编辑和查看学生信息。每个学生记录包括学生ID(学号)、姓名等基本信息。
2. 班级管理：管理员可以创建、删除、编辑和查看班级信息。班级记录包括班级名称、班级包含的学生等信息。

系统的架构环境基于纯粹的Java控制台项目，不依赖任何外部库或框架。数据存储采用简单的文本文件，每个实体（学生或班级）都存储在一个文本文件data.txt中。

运行环境要求Java运行时环境（JRE）已安装在管理员的计算机上。管理员可以通过命令行或集成开发环境（IDE）启动系统，并通过控制台进行交互操作。

系统的整体设计注重简单、高效和易用性，使管理员能够方便地管理学生信息和班级信息，而不需要复杂的操作或学习曲线。同时，采用本地存储数据的方式，使得系统具有良好的可移植性和稳定性，无需依赖外部数据库系统。

总体而言，这个Java控制台教务管理系统为管理员提供了一个便捷而实用的工具，可用于高效管理学生信息和班级信息，同时保持了简洁和易用的特点。

3. 功能性需求描述

3.1 班级管理

3.1.1 添加班级

用例ID	UC01
简述	管理员添加新的班级
执行者	管理员
前置条件	控制台中该管理系统已启动
后置条件	系统中成功添加了新的班级信息。 管理员可以选择继续操作或退出系统。
基本事件流	1. 管理员输入命令 ◦ 管理员在控制台中输入命令以选择相应菜单选项以执行添加班级操作。 2. 管理员输入班级信息 ◦ 管理员根据系统提示，在控制台中输入新班级的名称。 3. 系统验证信息 ◦ 系统验证管理员输入的班级信息，确保输入的信息格式正确且完整。 4. 系统确认添加成功 ◦ 系统向管理员发送确认消息，通知管理员新班级已成功添加。
异常事件流	1. 在步骤1，如果管理员在控制台输入的命令不正确或缺失必要信息，系统将提示管理员重新输入相关信息。 2. 在步骤3，如果系统检测到输入的班级信息格式不正确或与已存在的班级信息重复，系统将提示管理员重新输入相关信息。

3.1.2 删除班级

用例ID	UC02
------	------

简述	管理员可以删除班级，同时解除班级与学生的关联。
执行者	管理员
前置条件	控制台中该管理系统已启动，系统中至少存在一个班级。
后置条件	系统中成功删除了指定班级的信息。 系统同时删除了班级存在学生信息。 管理员可以选择继续操作或退出系统。
基本事件流	1. 管理员输入命令 ◦ 管理员在控制台中输入命令以选择相应菜单选项以执行删除班级操作。 2. 管理员选择要删除的班级 ◦ 管理员选择要删除的班级，并输入班级名称。 3. 系统删除班级信息 ◦ 系统删除选定的班级信息，并将与该班级相关联的学生信息一并删除。 4. 系统确认删除成功 ◦ 系统向管理员发送成功消息，通知管理员指定的班级已成功删除。
异常事件流	1. 在步骤1，如果管理员在控制台输入的命令不正确或缺失必要信息，系统将提示管理员重新输入相关信息。 2. 在步骤2，如果系统检测到输入的班级信息格式不正确或输入的班级信息不存在，系统将提示管理员重新输入相关信息。

3.1.3 查看班级信息

用例ID	UC03
简述	管理员可以查看每个班级的学生列表。
执行者	管理员
前置条件	控制台中该管理系统已启动，系统中至少存在一个班级。
后置条件	管理员成功查看了指定班级的信息。 管理员可以选择继续操作或退出系统。
基本事件流	1. 管理员输入命令 ◦ 管理员在控制台中输入命令以选择相应菜单选项以执行删除班级操作。 2. 系统展示信息 ◦ 系统展示所有班级以及班级中的学生信息

异常事件流	1. 在步骤1，如果管理员在控制台输入的命令不正确或缺失必要信息，系统将提示管理员重新输入相关信息。
-------	--

3.2 学生信息管理

3.2.1 添加学生

用例ID	UC04
简述	管理员可以添加学生，包括学号、姓名等信息。
执行者	管理员
前置条件	控制台中该管理系统已启动，系统中至少存在一个班级。
后置条件	管理员成功为该班级添加了学生信息。 管理员可以选择继续操作或退出系统。
基本事件流	- 管理员输入命令 - 管理员在控制台中输入命令以选择相应菜单选项以执行添加学生。 - 管理员输入班级信息 - 管理员在控制台中需要添加学生的班级信息。 - 管理员输入新学生信息 - 管理员在控制台中输入新学生的学号、姓名信息。 - 系统加入学生信息 - 系统为指定班级加入新学生的信息。 - 系统确认添加成功 - 系统向管理员发送成功消息，通知管理员向指定班级添加学生成功。
异常事件流	- 在步骤1，如果管理员在控制台输入的命令不正确或缺失必要信息，系统将提示管理员重新输入相关信息。 - 在步骤2，如果管理员在控制台输入的命令不正确或输入的班级不存在，系统将提示管理员重新输入相关信息。 - 在步骤3，如果管理员在控制台输入的命令不正确或输入的新学生的学号在该系统已经存在，系统将提示管理员重新输入相关信息

3.2.2 删除学生

用例ID	UC05
------	------

简述	管理员可以删除学生，同时解除学生与班级的关联。
执行者	管理员
前置条件	控制台中该管理系统已启动，系统中至少存在一个班级。
后置条件	管理员成功为删除了某班级的学生信息。 管理员可以选择继续操作或退出系统。
基本事件流	1. 管理员输入命令 ◦ 管理员在控制台中输入命令以选择相应菜单选项以执行删除学生。 2. 管理员输入班级信息 ◦ 管理员在控制台中需要删除学生的班级信息。 3. 管理员输入要删除的学生信息 ◦ 管理员在控制台中要删除学生的学号信息。 4. 系统删除学生信息 ◦ 系统为删除指定学生并且解除学生与班级的关联。 5. 系统确认删除成功 ◦ 系统向管理员发送成功消息，通知管理员删除指定学生成功。
异常事件流	1. 在步骤1，如果管理员在控制台输入的命令不正确或缺失必要信息，系统将提示管理员重新输入相关信息。 2. 在步骤2，如果管理员在控制台输入的命令不正确或输入的班级不存在，系统将提示管理员重新输入相关信息。 3. 在步骤3，如果管理员在控制台输入的命令不正确或输入的学生学号在该班级不存在，系统将提示管理员重新输入相关信息

3.3 数据存储管理

3.3.1 系统读入数据

用例ID	UC06
简述	管理员启动系统控制台，系统从本地文件读入班级学生信息。
执行者	管理员
前置条件	本地存储文件已存在。
后置条件	管理员成功启动教务管理系统。 教务管理系统成功加载数据。

基本事件流	1. 管理员启动系统 ◦ 管理员通过点击可执行文件启动教务管理系统。 2. 系统从本地文件读取数据 ◦ 系统查找本地数据文件，并通过本地数据进行系统初始化。 3. 系统成功启动 ◦ 控制台成功显示管理员可执行的操作指令提示。
异常事件流	1. 在步骤2，如果系统无法成功找到并加载数据文件，会为管理员新建数据文件并创建新的管理员角色。

3.3.2 系统保存数据

用例ID	UC07
简述	管理员关闭系统，系统自动保存数据信息。
执行者	管理员
前置条件	控制台中该管理系统已启动。本地存储文件已存在。
后置条件	管理员成功关闭并退出系统。 系统将班级和学生数据保存在本地文件。
基本事件流	1. 管理员输入命令 ◦ 管理员在控制台中输入命令以选择相应菜单选项以退出系统并保存信息。 2. 系统保存信息 ◦ 系统将班级信息以及学生信息写入本地数据文件进行保存 3. 系统确认数据保存成功 ◦ 系统向管理员发送成功消息，通知管理员删除数据保存成功。 4. 系统退出 ◦ 系统控制台关闭
异常事件流	1. 在步骤1，如果管理员在控制台输入的命令不正确或缺失必要信息，系统将提示管理员重新输入相关信息。 2. 在步骤2，系统向本地文件保存数据时出错，系统将向管理员报告数据保存发生错误的信息。

4. 系统架构设计

该教务管理系统采用Java语言开发，是一个控制台程序，通过控制台与管理员进行交互。系统的数据存储不依赖数据库，而是将数据存储在项目同一目录下的data.txt文件中。下面是该系统的整体架构设计：

4.1 模块划分

- 用户交互模块：负责与管理员进行交互，接收管理员的命令并执行相应操作，包括学生信息管理和班级管理等功能。
- 数据处理模块：负责读取、写入和处理数据，将数据存储在本地的data.txt文件中，并提供数据的读取、更新和删除功能。
- 业务逻辑模块：包含系统的业务逻辑，如学生信息管理和班级管理的具体操作，负责对管理员输入的命令进行解析并调用数据处理模块完成相应操作。

4.2 数据存储结构

数据存储采用简单的文本文件格式，每条记录使用特定的分隔符（如逗号或竖线）进行字段分隔。对于学生信息和班级信息，可以采用以下格式存储：

- 学生信息：每行存储一个学生的信息，各字段之间用逗号分隔，包括学生ID、姓名信息。
- 班级信息：每行存储一个班级的信息，各字段之间用逗号分隔，包括班级名称信息。

4.3 主要功能流程

- 启动程序：用户启动程序后，系统初始化，读取data.txt文件中的数据并加载到内存中，准备接受用户输入的命令。
- 用户交互：管理员通过控制台输入命令进行操作，系统接收命令并进行解析，根据命令调用相应的业务逻辑模块。
- 数据处理：业务逻辑模块调用数据处理模块读取或更新数据，操作完成后将结果写入data.txt文件中。
- 退出程序：管理员退出程序时，系统将当前内存中的数据写回data.txt文件，并进行必要的资源释放。

4.4 异常处理

系统需要进行异常处理，包括但不限于文件读写异常、命令解析异常等，保证系统的稳定性和可靠性。

4.5 扩展性考虑

为了提高系统的扩展性，可以考虑将各个功能模块封装成独立的类或模块，采用面向对象的设计原则，便于后续添加新功能或进行模块替换。

4.6 性能优化

在数据读写方面，可以采用缓存机制减少对文件的频繁读写，提高系统的性能和响应速度。

5. 类设计

5.1 Student 类

5.1.1 功能说明

Student 类表示一个学生，包含学生的学号和姓名信息。

5.1.2 数据成员

- studentId：学生的学号，String类型。
- name：学生的姓名，String类型。

5.1.3 方法(成员函数)

1. 构造函数：

- 输入参数：
 - studentId：学生的学号，String类型。
 - name：学生的姓名，String类型。
- 功能：创建一个新的学生对象。
- 边界值说明：无。

2. getId方法：

- 输入参数：无。
- 返回类型：String
- 功能：获取学生的学号。
- 边界值说明：无。

3. toString 方法：

- 输入参数：无。
- 返回类型：String
- 功能：返回包含学生学号和姓名的字符串表示形式。
- 边界值说明：无。

5.1.4 代码

```
1 class Student implements Serializable {
2     private String studentId;
3     private String name;
4
5     public Student(String studentId, String name) {
6         this.studentId = studentId;
7         this.name = name;
8     }
9
10    public String getStudentId() {
11        return studentId;
12    }
13
14    @Override
15    public String toString() {
16        return "学生ID: " + studentId + ", 姓名: " + name;
17    }
18 }
```

5.2 Classroom 类

5.2.1 功能说明

Classroom 类表示一个班级，包含班级名称和学生列表信息。

5.2.2 数据成员

- className：班级名称，String类型
- students：学生列表，ArrayList<Student>类型

5.2.3 成员函数

1. 构造方法：

- 输入参数：
 - className：班级名称，String类型。
- 功能：创建一个新的班级对象，并初始化学生列表为空。
- 边界值说明：无。

2. getClassName 方法：

- 输入参数：无
- 返回类型：String

- 功能：获取班级名称。
- 边界值说明：无。

3. addStudent 方法：

- 输入参数：
 - student：要添加的学生对象。
- 返回类型：void
- 功能：将指定的学生添加到班级的学生列表中。
- 边界值说明：无。

4. removeStudent 方法：

- 输入参数：
 - studentId：要删除的学生的学号，String类型。
- 返回类型：void
- 功能：根据学号从班级的学生列表中移除对应的学生。
- 边界值说明：无。

5. getStudents 方法：

- 输入类型：无
- 返回类型：ArrayList<Student>
- 功能：获取班级的学生列表。
- 边界值说明：无。

6. toString 方法：

- 输入类型：无
- 返回类型：String
- 功能：返回包含班级名称和学生列表信息的字符串表示形式。
- 边界值说明：无。

5.2.4 代码

```
1 class Classroom implements Serializable {
2     private String className;
3     private ArrayList<Student> students;
4
5     public Classroom(String className) {
6         this.className = className;
```

```

7         this.students = new ArrayList<>();
8     }
9
10    public String getClassName() {
11        return className;
12    }
13
14    public void addStudent(Student student) {
15        students.add(student);
16    }
17
18    public void removeStudent(String studentId) {
19        students.removeIf(student -> student.getStudentId().equals(studentId));
20    }
21
22    public ArrayList<Student> getStudents() {
23        return students;
24    }
25
26    @Override
27    public String toString() {
28        return "班级: " + className + "\n学生: " + students;
29    }
30 }

```

5.3 Admin 类

5.3.1 功能说明

Admin 类表示系统管理员，负责管理班级和学生信息。

5.3.2 数据成员

- classrooms：班级列表，ArrayList<Classroom>类型

5.3.3 成员函数

1. 构造方法：

- 输入参数：无
- 返回类型：void
- 功能： 创建一个新的管理员对象，并初始化班级列表为空。
- 边界值说明： 无。

2. ifExistClassroom 方法：

- 输入参数：
 - name: 班级名称, String类型。
- 返回类型: boolean
- 功能: 判断是否存在指定名称的班级。
- 边界值说明: 无。

3. ifExistStudent 方法:

- 参数：
 - studentId: 学生学号, String类型。
- 返回类型: boolean
- 功能: 判断是否存在指定学号的学生。
- 边界值说明: 无。

4. ifExistStudent 方法 (重载) :

- 参数：
 - className: 班级名称, String类型。
 - studentId: 学生学号, String类型。
- 返回类型: boolean
- 功能: 判断指定班级中是否存在指定学号的学生。
- 边界值说明: 无。

5. addClassroom 方法:

- 输入参数：
 - classroom: 要添加的班级对象, String类型。
- 返回类型: void
- 功能: 将指定班级添加到班级列表中。
- 边界值说明: 无。

6. removeClassroom 方法:

- 参数：
 - className: 要删除的班级的名称, String类型。
- 返回类型: void
- 功能: 根据名称从班级列表中移除对应的班级。
- 边界值说明: 无。

7. addStudentToClass 方法:

- 参数：
 - className：要添加学生的班级名称，String类型。
 - student：要添加的学生对象，String类型。
- 返回类型：void
- 功能：将指定学生添加到指定班级的学生列表中。
- 边界值说明：无。

8. removeStudentFromClass 方法：

- 参数：
 - className：要删除学生的班级名称，String类型。
 - studentId：要删除的学生的学号，String类型。
- 返回类型：void
- 功能：从指定班级的学生列表中移除指定学号的学生。
- 边界值说明：无。

9. displayClassrooms 方法：

- 输入参数：无
- 返回类型：void
- 功能：显示所有班级的信息。
- 边界值说明：无。

10. saveToFile 方法：

- 输入参数：无
- 返回类型：void
- 功能：将管理员对象保存到文件中。
- 边界值说明：无。

11. loadFromFile 静态方法：

- 输入参数：无
- 返回类型：Admin
- 功能：从文件中加载管理员对象。
- 边界值说明：无。

5.3.4 代码

```
1 class Admin implements Serializable {
2     private ArrayList<Classroom> classrooms;
3
4     public Admin() {
5         this.classrooms = new ArrayList<>();
6     }
7
8     public boolean ifExistClassroom(String name){
9         for(Classroom c : classrooms){
10             if(name.equals(c.getClassName())) return true;
11         }
12         return false;
13     }
14
15     public boolean ifExistStudent(String studentId){
16         for(Classroom c : classrooms){
17             for(Student s : c.getStudents()){
18                 if(studentId.equals(s.getStudentId())){
19                     return true;
20                 }
21             }
22         }
23         return false;
24     }
25
26     public boolean ifExistStudent(String className, String studentId){
27         for (Classroom classroom : classrooms) {
28             if (classroom.getClassName().equals(className)) {
29                 for (Student s : classroom.getStudents()) {
30                     if (studentId.equals(s.getStudentId())) {
31                         return true;
32                     }
33                 }
34                 break;
35             }
36         }
37         return false;
38     }
39
40     public void addClassroom(Classroom classroom) {
41         classrooms.add(classroom);
42     }
43
44     public void removeClassroom(String className) {
45         classrooms.removeIf(c -> c.getClassName().equals(className));
46     }
47 }
```

```
48     public void addStudentToClass(String className, Student student) {
49         for (Classroom classroom : classrooms) {
50             if (classroom.getClassName().equals(className)) {
51                 classroom.addStudent(student);
52                 break;
53             }
54         }
55     }
56
57     public void removeStudentFromClass(String className, String studentId) {
58         for (Classroom classroom : classrooms) {
59             if (classroom.getClassName().equals(className)) {
60                 classroom.removeStudent(studentId);
61                 break;
62             }
63         }
64     }
65
66     public void displayClassrooms() {
67         for (Classroom classroom : classrooms) {
68             System.out.println(classroom);
69         }
70     }
71
72     public void saveToFile() {
73         try (ObjectOutputStream outputStream = new ObjectOutputStream(new
74             FileOutputStream("data.txt"))) {
75             outputStream.writeObject(this);
76             System.out.println("信息已保存到 data.txt 文件。");
77         } catch (IOException e) {
78             System.out.println("保存信息时发生错误: " + e.getMessage());
79         }
80
81     public static Admin loadFromFile() {
82         try (ObjectInputStream inputStream = new ObjectInputStream(new
83             FileInputStream("data.txt"))) {
84             return (Admin) inputStream.readObject();
85         } catch (IOException | ClassNotFoundException e) {
86             System.out.println("第一次使用该系统, 以为您自动创建管理员角色和data.txt
87             文件");
88             return new Admin();
89         }
90     }
```

5.4 Menu 类

5.4.1 功能说明

Menu 类表示控制台菜单，负责用户交互和调用 Admin 类的方法。

5.4.2 数据成员

- reader：数据读取缓冲区，类型为BufferedReader
- admin：管理员角色，类型为Admin

5.4.3 成员函数

1. 构造方法：

- 输入参数：
 - admin：Admin 类的实例，用于执行相关操作。
- 返回类型：void
- 功能：创建一个新的菜单对象，并初始化 BufferedReader 和 Admin 对象。
- 边界值说明：无。

2. displayMenu 方法：

- 输入参数：无
- 返回类型：void
- 功能：显示菜单选项，并根据用户的选择执行相应的操作，直到用户选择退出。
- 边界值说明：无。

3. addClassroom 方法：

- 输入参数：无
- 返回类型：void
- 功能：添加班级操作。
- 边界值说明：无。

4. removeClassroom 方法：

- 输入参数：无
- 返回类型：void
- 功能：删除班级操作。
- 边界值说明：无。

5. addStudentToClass 方法：

- 输入参数：无
- 返回类型：void
- 功能： 将学生添加到班级操作。
- 边界值说明： 无。

6. removeStudentFromClass 方法：

- 输入参数：无
- 返回类型：void
- 功能： 从班级中删除学生操作。
- 边界值说明： 无。

7. displayClassrooms 方法：

- 输入参数：无
- 返回类型：void
- 功能： 显示所有班级信息操作。
- 边界值说明： 无。

8. saveAndExit 方法：

- 输入参数：无
- 返回类型：void
- 功能： 保存数据并退出程序操作。
- 边界值说明： 无。

5.4.4 代码

```
1 class Menu {
2     private BufferedReader reader;
3     private Admin admin;
4
5     public Menu(Admin admin) {
6         this.reader = new BufferedReader(new InputStreamReader(System.in));
7         this.admin = admin;
8     }
9
10    public void displayMenu() {
11        int choice = -1; // 初始化 choice
12        do {
13            System.out.println("1. 添加班级");
14            System.out.println("2. 删除班级");
```

```
15         System.out.println("3. 将学生添加到班级");
16         System.out.println("4. 从班级中删除学生");
17         System.out.println("5. 显示班级信息");
18         System.out.println("0. 退出并保存");
19         System.out.print("请输入您的选择: ");
20         try {
21             choice = Integer.parseInt(reader.readLine());
22         } catch (IOException | NumberFormatException e) {
23             System.out.println("输入无效, 请重新输入。");
24             continue;
25         }
26
27         switch (choice) {
28             case 1:
29                 addClassroom();
30                 break;
31             case 2:
32                 removeClassroom();
33                 break;
34             case 3:
35                 addStudentToClass();
36                 break;
37             case 4:
38                 removeStudentFromClass();
39                 break;
40             case 5:
41                 displayClassrooms();
42                 break;
43             case 0:
44                 saveAndExit();
45                 break;
46             default:
47                 System.out.println("无效的选择, 请重新输入。");
48         }
49     } while (choice != 0);
50 }
51
52 private void addClassroom() {
53     System.out.print("请输入班级名称: ");
54     try {
55         String className = reader.readLine();
56         if (admin.ifExistClassroom(className)) {
57             throw new ClassRepeatException();
58         }
59         admin.addClassroom(new Classroom(className));
60         System.out.println("班级添加成功。");
61     } catch (IOException e) {
```

```
62         System.out.println("输入无效, 请重新输入。");
63     } catch (ClassRepeatException e) {
64         System.out.println("已经存在该班级, 请重新输入。");
65     }
66 }
67
68 private void removeClassroom() {
69     System.out.print("请输入班级名称: ");
70     try {
71         String className = reader.readLine();
72         if(!admin.ifExistClassroom(className)){
73             throw new ClassNotExistException();
74         }
75         admin.removeClassroom(className);
76         System.out.println("班级删除成功。");
77     } catch (IOException e) {
78         System.out.println("输入无效, 请重新输入。");
79     } catch (ClassNotExistException e) {
80         System.out.println("输入的班级不存在, 请重新输入。");
81     }
82 }
83
84 private void addStudentToClass() {
85     try {
86         System.out.print("请输入班级名称: ");
87         String className = reader.readLine();
88         if(!admin.ifExistClassroom(className)){
89             throw new ClassNotExistException();
90         }
91         System.out.print("请输入学生ID: ");
92         String studentId = reader.readLine();
93         if(admin.ifExistStudent(studentId)){
94             throw new StudentRepeatException();
95         }
96         System.out.print("请输入学生姓名: ");
97         String studentName = reader.readLine();
98         admin.addStudentToClass(className, new Student(studentId,
99 studentName));
100         System.out.println("学生成功添加到班级。");
101     } catch (IOException e) {
102         System.out.println("输入无效, 请重新输入。");
103     } catch (StudentRepeatException e) {
104         System.out.println("该学生ID已经存在, 请重新输入。");
105     } catch (ClassNotExistException e) {
106         System.out.println("输入的班级不存在, 请重新输入。");
107     }
108 }
```

```

108
109     private void removeStudentFromClass() {
110         try {
111             System.out.print("请输入班级名称: ");
112             String className = reader.readLine();
113             if(!admin.ifExistClassroom(className)){
114                 throw new ClassNotExistException();
115             }
116             System.out.print("请输入学生ID: ");
117             String studentId = reader.readLine();
118             if(!admin.ifExistStudent(className,studentId)){
119                 throw new StudentNotExistException();
120             }
121             admin.removeStudentFromClass(className, studentId);
122             System.out.println("学生成功从班级中移除。");
123         } catch (IOException e) {
124             System.out.println("输入无效, 请重新输入。");
125         } catch (StudentNotExistException e) {
126             System.out.println("该班级不存在该学生ID, 请重新输入。");
127         } catch (ClassNotExistException e) {
128             System.out.println("输入的班级不存在, 请重新输入。");
129         }
130     }
131
132     private void displayClassrooms() {
133         admin.displayClassrooms();
134     }
135
136     private void saveAndExit() {
137         admin.saveToFile();
138         System.out.println("程序已退出并信息已保存到 data.txt 文件。");
139     }
140 }

```

5.5 自定义异常类

5.5.1 功能说明

自定义异常类用于处理程序中的异常情况，包括班级不存在、班级重复、学生不存在和学生重复等情况。

5.5.2 异常类

1. ClassNotExistException 类:

- 功能：班级不存在异常。

- 边界值说明：无。
2. ClassRepeatException 类：
- 功能： 班级重复异常。
 - 边界值说明： 无。
3. StudentNotExistException 类：
- 功能： 学生不存在异常。
 - 边界值说明： 无。
4. StudentRepeatException 类：
- 功能： 学生重复异常。
 - 边界值说明： 无。

5.5.3 代码

```
1 class ClassNotExistException extends Exception {
2     public ClassNotExistException() {
3         super();
4     }
5 }
6
7 class ClassRepeatException extends Exception {
8     public ClassRepeatException() {
9         super();
10    }
11 }
12
13 class StudentNotExistException extends Exception{
14     public StudentNotExistException() {
15         super();
16     }
17 }
18
19 class StudentRepeatException extends Exception{
20     public StudentRepeatException() {
21         super();
22     }
23 }
```

6. 非功能性需求

6.1 负载性能

该系统应能够处理至少1000个班级和每个班级至少100个学生的信息。在高负载情况下，系统的响应时间应保持在1秒以内，吞吐量应支持每秒至少100个用户并发请求。在设计和开发过程中，需要进行性能测试，确保系统在预期负载下的性能表现。

6.2 兼容性

该系统应能够在主流的操作系统（如Windows、Linux、macOS等）上正常运行，并且与常用的Java虚拟机版本兼容，包括JDK 8及以上版本。

6.3 本地化

系统应支持至少2种语言的本地化，包括英语、中文等。用户界面文本、错误提示等应能够根据用户所选的语言进行动态切换，并且应提供易于扩展和更新的本地化资源文件。系统应遵循当地的法律法规和文化习惯，确保用户体验的合法性和舒适性。

6.4 安全性

系统应具备严格的安全性措施，包括但不限于：

- 数据加密：对于敏感数据在传输和存储过程中进行AES加密保护，确保数据的机密性和完整性。
- 安全审计：记录和监控系统的操作行为，包括登录、数据访问和权限变更等，及时发现和应对可能存在的安全威胁和漏洞。
- 漏洞修复：及时更新和修复系统中发现的安全漏洞和问题，保持系统的安全性和稳定性。